

NewtonX ADK API リファレンス（NewtonX エージェント開発キット）

概要

NewtonX ADKは、NewtonX APIを簡単に利用できるPythonライブラリです。認証、チャット、アシスタント、ファイルアップロードなどの機能を提供します。

クラス一覧

NewtonXClient

NewtonX APIとの通信を行うメインクライアントクラスです。

初期化

```
from newtonx_adk import NewtonXClient, ConfigManager

# 設定管理クラスを初期化
config_manager = ConfigManager()

# クライアントを初期化
client = NewtonXClient(config_manager)
```

メソッド

authenticate() -> bool

認証を実行します。

```
if client.authenticate():
    print("認証が成功しました！")
else:
    print("認証に失敗しました。")
```

get_user_info() -> Optional[Dict]

ユーザー情報を取得します。

```
user_info = client.get_user_info()
if user_info:
    print(f"ユーザー名: {user_info.get('name')}")
    print(f"メール: {user_info.get('email')}")
```

戻り値:

- Optional[Dict]:以下の形式でユーザ情報がかえります。

```
{
    'name': 'セラク 太郎',
    'email': '*****@seraku.co.jp',
    'company_id': 1,
    'organization_admin': False,
    'system_admin': False,
    'role': {'knowledge': None},
    'departments': []
}
or
None
```

get_assistants() -> List[Dict]

アシスタント一覧を取得します。

```
assistants = client.get_assistants()
for assistant in assistants:
    print(f"アシスタント: {assistant['name']} (ID: {assistant['uid']})")
```

戻り値:

- List[Dict]:Listで返されるアシスタント数はユーザによって異なります

```
[
    {
        'id': 73,
        'uid': '03a668d0-e8ea-4dcd-bbdb-43d81832e808',
        'uuid': '03a668d0-e8ea-4dcd-bbdb-43d81832e808',
        'name': '高性能 (GPT-4o) ',
        'description': '自然な会話が得意で、文章作成や壁打ちに最適です。',
        'icon_color': '#6495ED',
        'has_avatar': False,
        'type': 'system',
        'is_recent': False,
        'gpt-3.5': False,
        'questions': [],
    }
]
```

```

    'me': {'role': 'viewer', 'joined_at': None},
    'users_count': 0,
    'files_count': 0,
    'is_favorite': True,
    'favorite_add_time': '2025-07-03 01:54:44'
  },
    * * * 省略 * * *
  {
    'id': 1525,
    'uid': 'a7251d55-a886-4726-8768-7fa858e1d697',
    'uuid': 'a7251d55-a886-4726-8768-7fa858e1d697',
    'name': 'NewtonX ADK',
    'description': 'NewtonXのADKの使い 方をお手伝いするアシスタントです。\\nADKの
ver 0.10.3に対応しています。',
    'icon_color': '#90D126',
    'has_avatar': True,
    'type': 'company',
    'is_recent': False,
    'gpt-3.5': True,
    'questions':
      [
        {
          'uid': '29e5e53d-707b-46b7-b6a1-9c0eba3e01d4',
          'uuid': '29e5e53d-707b-46b7-b6a1-9c0eba3e01d4',
          'question': 'ADKのインストール方法は？',
          'updated_at': '2025-10-03 10:37:24'
        },
        * * * 省略 * * *
        {
          'uid': 'e82174bd-437e-4428-b555-65c82213e79f',
          'uuid': 'e82174bd-437e-4428-b555-65c82213e79f',
          'question': 'Auto Runnerサンプルの使い方を教えてください',
          'updated_at': '2025-08-01 04:14:19'
        }
      ],
    'me': {'role': 'viewer', 'joined_at': '2025-08-01 04:13:07'},
    'users_count': 2,
    'files_count': 8,
    'is_favorite': False,
    'favorite_add_time': None
  }
]

```

get_chats(page: int = 1, page_size: int = 20, search_query: Optional[str] = None) -> List[Dict]

チャット一覧を取得します。

```

# 全チャットを取得
chats = client.get_chats()

# 検索クエリでフィルタリング

```

```
search_results = client.get_chats(search_query="会議")

# ページネーション
page2_chats = client.get_chats(page=2, page_size=10)
```

戻り値:

- List[Dict]: (Listで返されるチャット情報の数はユーザによって異なります)

```
[
  {
    'id': '0c2f278a-81d8-44f3-8bf6-bd273f8dfb23',
    'title': 'VSCodeのtasks.jsonに現在のファイルを引数で実行する タスクの追加方
法',
    'folder_id': None,
    'share': False,
    'chat_type': 'normal',
    'assistant':
      {
        'uuid': '20e5f668-4850-482f-b6c0-4ae2e5a2430b',
        'type': 'system',
        'name': '高速 (GPT-5 mini) ',
        'model': 'gpt-5-mini'
      }
  },
  * * * 省略 * * *
  {
    'id': '936b4a4b-bbbb-4948-b000-396413b2d6eb',
    'title': 'チャット選択機能の追加と画像解析プログラムの改造',
    'folder_id': None,
    'share': False,
    'chat_type': 'knowledge',
    'assistant':
      {
        'uuid': 'a7251d55-a886-4726-8768-7fa858e1d697',
        'type': 'company',
        'name': 'NewtonX ADK',
        'model': None
      }
  }
]
```

create_chat(assistant_uid: str, title: Optional[str] = None, folder_uid: Optional[str] = None) -> Optional[str]

チャットを作成します。

```
chat_uid = client.create_chat(
    assistant_uid="assistant_uid_here",
```

```

    title="新しいチャット",
    folder_uid="folder_uid_here" # オプション
)

```

戻り値:

- Optional[str]: 生成したチャットの'uid','uuid'の値（同値）を返します。

get_chat(chat_uid: str) -> Optional[Dict]

チャット詳細を取得します。

```

chat_detail = client.get_chat("chat_uid_here")
if chat_detail:
    print(f"チャットタイトル: {chat_detail['title']}")
    print(f"アシスタント: {chat_detail['assistant_name']}")

```

戻り値:

- Optional[str]: 生成したチャットの詳細情報を返します。

```

{
  'id': '4160db24-7ff7-4532-a76a-04406db9a355',
  'chat_uuid': '4160db24-7ff7-4532-a76a-04406db9a355',
  'title': 'バンプコーディングの概要',
  'share': False,
  'chat_type': 'normal',
  'assistant':
  {
    'uid': '03a668d0-e8ea-4dcd-bbdb-43d81832e808',
    'uuid': '03a668d0-e8ea-4dcd-bbdb-43d81832e808',
    'type': 'system',
    'name': '高性能 (GPT-4o) ',
    'model': 'gpt-4o'},
    'messages':
    [
      {
        'role': 'user',
        'content': 'バンプコーディングとは',
        'model': 'gpt-4o',
        'chat_order': 1,
        'parent_order': 0,
        'metadata':
          {
            'mode': 'web',
            'finish_reason': 'stop',
            'references':
              [
                {

```

```

        'url':
'https://www.technologyreview.jp/s/359884/what-is-vibe-coding-exactly/',
        'title': 'バイブコーディングとは何か？ AIに
「委ねる」プログラミング新手法'
    },
    * * * 省略 * * *
    {
        'url':
'https://zenn.dev/fugafuga/articles/9f999869812c17', 'title': 'バイブコーディングと
いう地獄 - Zenn'
    }
]
}
},
{
    'role': 'assistant',
    'content': 'バイブコーディングとは、AI（人工知能）を活用してソ
フトウェアを開発する新しい手法の一つです。・省略・（md形式)',
    'model': 'gpt-4o',
    'chat_order': 2,
    'parent_order': 1,
    'metadata':
    {
        'mode': 'web',
        'finish_reason': 'stop',
        'references':
        [
            {
                'url':
'https://www.technologyreview.jp/s/359884/what-is-vibe-coding-exactly/',
                'title': 'バイブコーディングとは何か？
AIに「委ねる」プログラミング新手法'
            },
            * * * 省略 * * *
            {
                'url':
'https://zenn.dev/fugafuga/articles/9f999869812c17', 'title': 'バイブコーディングと
いう地獄 - Zenn'
            }
        ]
    }
}
]
}

```

send_message(chat_uid: str, message: str, knowledge_search: bool = False, web_search: bool = True, image_ids: Optional[List[str]] = None, document_ids: Optional[List[str]] = None, audio_file_path: Optional[str] = None, parent_order: int = 0) -> Optional[str]

メッセージを送信します。

```
# 基本的なメッセージ送信
response = client.send_message(
    chat_uid="chat_uid_here",
    message="こんにちは！"
)

# 文脈を指定してメッセージ送信（前のメッセージの続きとして送信）
response = client.send_message(
    chat_uid="chat_uid_here",
    message="続きを教えて",
    parent_order=5
)

# 検索設定を指定
response = client.send_message(
    chat_uid="chat_uid_here",
    message="最新のAI技術について教えてください",
    web_search=True,
    knowledge_search=False
)

# 画像付きメッセージ
response = client.send_message(
    chat_uid="chat_uid_here",
    message="この画像を分析してください",
    image_ids=["image_id_1", "image_id_2"]
)

# 音声付きメッセージ
response = client.send_message(
    chat_uid="chat_uid_here",
    message="この音声を要約してください",
    audio_file_path="/path/to/sample.wav"
)
```

パラメータ:

- `chat_uid` (str): チャットのUID
- `message` (str): 送信するメッセージ
- `knowledge_search` (bool, デフォルト: False): ナレッジ検索を有効にするか
- `web_search` (bool, デフォルト: True): ウェブ検索を有効にするか
- `image_ids` (Optional[List[str]]): 画像IDのリスト（画像付きメッセージの場合）
- `document_ids` (Optional[List[str]]): ドキュメントIDのリスト（ドキュメント参照の場合）
- `audio_file_path` (Optional[str]): 音声ファイルのパス（音声付きメッセージの場合）
- `parent_order` (int, デフォルト: 0): 親メッセージの順序（スレッドの文脈を維持する場合に指定）

音声ファイルの仕様:

- 対応ファイル形式: `.wav`, `.mp3`, `.aiff`, `.aac`, `.ogg`, `.flac`
- 最大ファイルサイズ: 15MB

- `audio_file_path` を指定した場合、リクエストは `multipart/form-data` 形式で送信されます
- `image_ids` と `audio_file_path` は同時に指定できません（相互排他）

戻り値:

- `Optional[str]`: アシスタントの応答テキスト（SSEストリーミングの場合は処理済みテキスト）。エラー時は例外が発生します。

`delete_chat(chat_uid: str) -> bool`

チャットを削除します。

```
success = client.delete_chat("chat_uid_here")
if success:
    print("チャットが削除されました")
```

`update_chat_title(chat_uid: str, title: str) -> bool`

チャットタイトルを更新します。

```
success = client.update_chat_title("chat_uid_here", "新しいタイトル")
if success:
    print("チャットタイトルが更新されました")
```

`create_folder(name: str) -> Optional[str]`

フォルダを作成します。

```
folder_uid = client.create_folder("新しいフォルダ")
if folder_uid:
    print(f"フォルダが作成されました: {folder_uid}")
```

`get_folders() -> List[Dict]`

フォルダー一覧を取得します。

```
folders = client.get_folders()
for folder in folders:
    print(f"フォルダ: {folder['name']} (ID: {folder['id']})")
```

戻り値:

- `List[Dict]`: フォルダーの情報をリストで返します。


```
[
  {
    'type': 'folder',
    'id': 5656,
    'name': '領収書読み取り'
  },
  * * * 省略 * * *
  {
    'type': 'folder',
    'id': 6316,
    'name': 'AI最新技術'
  }
]
```

delete_folder(folder_id: int) -> bool

フォルダを削除します。フォルダ内のチャットも削除されます。

```
success = client.delete_folder(5656)
if success:
    print("フォルダが削除されました")
```

パラメータ:

- **folder_id** (int): 削除するフォルダのID

戻り値:

- **bool**: 削除が成功した場合 **True**、失敗した場合 **False**

move_chat_to_folder(chat_uid: str, folder_uid: str) -> bool

チャットをフォルダに移動します。

```
success = client.move_chat_to_folder("chat_uid_here", "folder_uid_here")
if success:
    print("チャットがフォルダに移動されました")
```

create_chat_in_folder(assistant_uid: str, folder_uid: str, title: Optional[str] = None) -> Optional[str]

フォルダ内にチャットを作成します（内部的には作成→移動の手順）。

```
chat_uid = client.create_chat_in_folder(
    assistant_uid="assistant_uid_here",
    folder_uid="folder_uid_here",
```

```
        title="フォルダ内チャット"
    )
    if chat_uid:
        print(f"フォルダ内にチャットを作成しました: {chat_uid}")
```

create_chat_in_folder_by_name(assistant_uid: str, folder_name: str, title: Optional[str] = None, create_if_missing: bool = True)
-> Optional[str]

フォルダ名を指定してチャットを作成します。指定名のフォルダが存在すればそれを使用し、なければ作成してからチャットを移動します。フォルダ作成直後の反映遅延に対して自動的に待機とリトライを行います。

```
chat_uid = client.create_chat_in_folder_by_name(
    assistant_uid="assistant_uid_here",
    folder_name="分析レポート",
    title="2025-09-実験記録"
)
if chat_uid:
    print(f"フォルダ名指定でチャットを作成しました: {chat_uid}")
```

get_folder_chats(folder_uid: str) -> List[Dict]

フォルダ内のチャット一覧を取得します。

```
folder_chats = client.get_folder_chats("folder_uid_here")
for chat in folder_chats:
    print(f"チャット: {chat['title']}")
```

戻り値:

- List[Dict]: 指定したフォルダ内にあるチャット情報をList形式で返します。

```
[
  {
    'id': '4160db24-7ff7-4532-a76a-04406db9a355',
    'title': 'バイブコーディングの概要',
    'folder_id': 6316,
    'share': False,
    'chat_type': 'normal',
    'assistant':
      {
        'uuid': '03a668d0-e8ea-4dcd-bbdb-43d81832e808',
        'type': 'system',
        'name': '高性能 (GPT-4o) ',
        'model': 'gpt-4o'
      }
  }
]
```

```

    }
    * * * 省略 * * *
    {
        'id': '33591819-758b-455b-bf7a-e7896a9c49f0',
        'title': 'AIを使ったプログラミングの呼称',
        'folder_id': 6316,
        'share': False,
        'chat_type': 'normal',
        'assistant':
            {
                'uuid': '03a668d0-e8ea-4dcd-bbdb-43d81832e808',
                'type': 'system',
                'name': '高性能 (GPT-4o) ',
                'model': 'gpt-4o'
            }
    }
]

```

upload_image(chat_uid: str, file_path: str, file_name: Optional[str] = None) -> Optional[str]

画像ファイルをアップロードします。

```

image_id = client.upload_image(
    chat_uid="chat_uid_here",
    file_path="path/to/image.jpg",
    file_name="custom_name.jpg" # オプション
)
if image_id:
    print(f"画像がアップロードされました: {image_id}")

```

upload_document(chat_uid: str, file_path: str, file_name: Optional[str] = None) -> bool

ドキュメントファイルをアップロードします。

```

success = client.upload_document(
    chat_uid="chat_uid_here",
    file_path="path/to/document.pdf",
    file_name="custom_name.pdf" # オプション
)
if success:
    print("ドキュメントがアップロードされました")

```

add_assistant_knowledge(assistant_uid: str, file_path: str, file_name: Optional[str] = None) -> Optional[str]

アシスタントにナレッジ（RAG用ファイル）を登録します。対応形式は `.docx`, `.xls`, `.xlsx`, `.pptx`, `.pdf`, `.txt` で、16MB以下のファイルのみアップロードできます。

戻り値:

- 戻り値はファイルID（取得できない環境では `None`）。

```
file_uid = client.add_assistant_knowledge(  
    assistant_uid="assistant_uid_here",  
    file_path="path/to/knowledge.pdf",  
)  
print(f"登録されたファイルID: {file_uid}")
```

`delete_assistant_knowledge(assistant_uid: str, file_uid: str) -> bool`

アシスタントから登録済みナレッジを削除します。成功時 `True` を返します。

```
ok = client.delete_assistant_knowledge(  
    assistant_uid="assistant_uid_here",  
    file_uid="file_uid_here",  
)  
print("削除成功" if ok else "削除失敗")
```

`get_assistant_knowledge(assistant_uid: str) -> List[Dict]`

指定アシスタントに登録済みのナレッジ一覧を取得します。返却形式の差異を吸収し、配列または `{files|data|items|results}` キーのいずれかに対応します。

```
files = client.get_assistant_knowledge("assistant_uid_here")  
for f in files:  
    print(f"ID={f.get('uid') or f.get('id')}, name={f.get('name')}")
```

戻り値:

- `List[Dict]`: 指定したアシスタント内にあるファイル情報をList形式で返します。

```
[  
    {  
        'uid': '63348ad3-5e3a-4575-a531-d4e23eb599d6',  
        'uuid': '63348ad3-5e3a-4575-a531-d4e23eb599d6',  
        'name': 'API_REFERENCE.pdf',  
        'size': 253730,  
        'status': 'Connected',  
        'uploaded_at': '2025-11-11T00:53:18Z',  
        'writer': 'セラク 太郎'  
    },  
]
```

省略

]

get_model_status() -> Dict[str, bool]

モデルステータスを取得します。

```
status = client.get_model_status()
for model, available in status.items():
    print(f"{model}: {'利用可能' if available else '利用不可'}")
```

get_company_info() -> Optional[Dict]

会社情報を取得します。

```
company_info = client.get_company_info()
if company_info:
    print(f"会社名: {company_info.get('name')}")
    print(f"ドメイン: {company_info.get('domain')}")
```

戻り値:

- List[Dict]: 会社情報をList形式で返します。

```
{'name': '株式会社セラク'}
```

upload_images(chat_uid: str, file_paths: List[str]) -> List[str]

複数の画像を結合して1枚のコラージュ画像を生成し、単一画像としてアップロードします。戻り値は生成画像に対応する1つのUIDを要素に持つ配列になります。

```
image_ids = client.upload_images(chat_uid, ["/path/to/a.png", "/path/to/b.jpg"])
# ["uid_collage"]
```

注意: 現時点のLLM処理が単一画像を前提としているための仕様です。将来的にマルチ画像入力が有効化されたら、ADK側の挙動を変更する可能性があります。

send_message_with_images(chat_uid: str, message: str, image_file_paths: Optional[List[str]] = None, knowledge_search: bool = False, web_search: bool = True, document_ids: Optional[List[str]] = None, parent_order: int = 0) -> Optional[str]

画像アップロードとメッセージ送信を一括で行います。`image_file_paths` に指定した画像をアップロードし、その返却 `UID` を `metadata.image_ids` として設定してメッセージを送信します。

```
resp = client.send_message_with_images(  
    chat_uid=chat_uid,  
    message="画像を解析してください",  
    image_file_paths=["/path/a.png", "/path/b.jpg"],  
    knowledge_search=False,  
    web_search=True,  
)
```

ConfigManager

設定管理クラスです。

初期化

```
from newtonx_adk import ConfigManager  
  
# デフォルト設定ファイルを使用  
config_manager = ConfigManager()  
  
# カスタム設定ファイルを指定  
config_manager = ConfigManager("custom_config.json")
```

メソッド

get_config() -> ADKConfig

設定を取得します。

```
config = config_manager.get_config()  
print(f"API Base URL: {config.api_base_url}")  
print(f"Timeout: {config.timeout}")
```

update_config(kwargs)**

設定を更新します。

```
config_manager.update_config(  
    api_base_url="https://api.newtonx.com",  
    timeout=30,  
    max_retries=3  
)
```

set_credentials(client_id: str, tenant_id: str)

認証情報を設定します（PKCE利用時はクライアントシークレット不要）。

```
config_manager.set_credentials(  
    client_id="your_client_id",  
    tenant_id="your_tenant_id"  
)
```

AuthManager

認証管理クラスです。

初期化

```
from newtonx_adk import AuthManager, ConfigManager  
  
config_manager = ConfigManager()  
auth_manager = AuthManager(config_manager)
```

メソッド

is_authenticated() -> bool

認証済みかどうかを確認します。

```
if auth_manager.is_authenticated():  
    print("認証済みです")  
else:  
    print("認証が必要です")
```

authenticate() -> bool

手動認証を実行します。

```
if auth_manager.authenticate():  
    print("認証が成功しました")  
else:  
    print("認証に失敗しました")
```

authenticate_auto() -> bool

自動認証を実行します（保存されたトークンを使用）。

```
if auth_manager.authenticate_auto():  
    print("自動認証が成功しました")  
else:  
    print("自動認証に失敗しました")
```

get_headers() -> Dict[str, str]

認証ヘッダーを取得します。

```
headers = auth_manager.get_headers()  
# {'Authorization': 'Bearer token_here', 'Content-Type': 'application/json'}
```

logout()

ログアウトします。

```
auth_manager.logout()  
print("ログアウトしました")
```

例外クラス

NewtonXError

ADKの基本例外クラスです。

AuthenticationError

認証関連の例外です。

APIError

API通信関連の例外です。

```
from newtonx_adk import NewtonXError, AuthenticationError, APIError  
  
try:  
    response = client.send_message(chat_uid, "テストメッセージ")  
except AuthenticationError as e:  
    print(f"認証エラー: {e}")  
except APIError as e:  
    print(f"APIエラー: {e}")  
    print(f"ステータスコード: {e.status_code}")  
    print(f"レスポンス: {e.response}")
```



```
except NewtonXError as e:
    print(f"その他のエラー: {e}")
```

設定

ADKConfig

設定データクラスです。

```
@dataclass
class ADKConfig:
    host: str = "" # Host-first: 例 seraku.newton-x.net
    api_base_url: str = "https://seraku.newton-x.net/api" # host から自動導出
    client_id: str = ""
    tenant_id: str = ""
    redirect_uri: str = "http://localhost:8400"
    token_file: str = "~/.newtonx/tokens.json"
    config_file: str = "~/.newtonx/config.json"
    timeout: int = 30
    max_retries: int = 3
```

使用例

基本的な使用例

```
from newtonx_adk import NewtonXClient, ConfigManager

# 設定とクライアントを初期化 (PKCE前提)
config_manager = ConfigManager()
config_manager.set_credentials("your_client_id", "your_tenant_id")

client = NewtonXClient(config_manager)

# 認証
if client.authenticate():
    # アシスタント一覧を取得
    assistants = client.get_assistants()

    # フォルダを作成
    folder_uid = client.create_folder("テストフォルダ")

    # フォルダ内にチャットを作成
    chat_uid = client.create_chat_in_folder(
        assistant_uid=assistants[0]['uid'],
        folder_uid=folder_uid,
        title="テストチャット"
    )

    # メッセージを送信
```

```
response = client.send_message(chat_uid, "こんにちは!")
print(f"応答: {response}")
```

ファイルアップロード例

```
# 画像ファイルをアップロード
image_id = client.upload_image(chat_uid, "receipt.jpg")

# 画像付きメッセージを送信
response = client.send_message(
    chat_uid=chat_uid,
    message="この領収書を分析してください",
    image_ids=[image_id]
)
```

エラーハンドリング例

```
try:
    response = client.send_message(chat_uid, "テストメッセージ")
except AuthenticationError:
    print("認証が必要です。認証を実行してください。")
    client.authenticate()
    response = client.send_message(chat_uid, "テストメッセージ")
except APIError as e:
    print(f"APIエラー: {e}")
except Exception as e:
    print(f"予期しないエラー: {e}")
```